AppsClub

2026/2027

Git Course



Session 2: Branches & Merging

with Soufiane

Course Plan

01
The Problem

02
Git Branches

03
Switching & HEAD

04
Merging Strategies

05
Merge Conflicts

06
Challenge & Demo

AppsClub

One Project, Multiple Tasks

A Developer A

Building login page.

B Developer B

Creating contact form.

C Developer C

Fixing urgent bug.



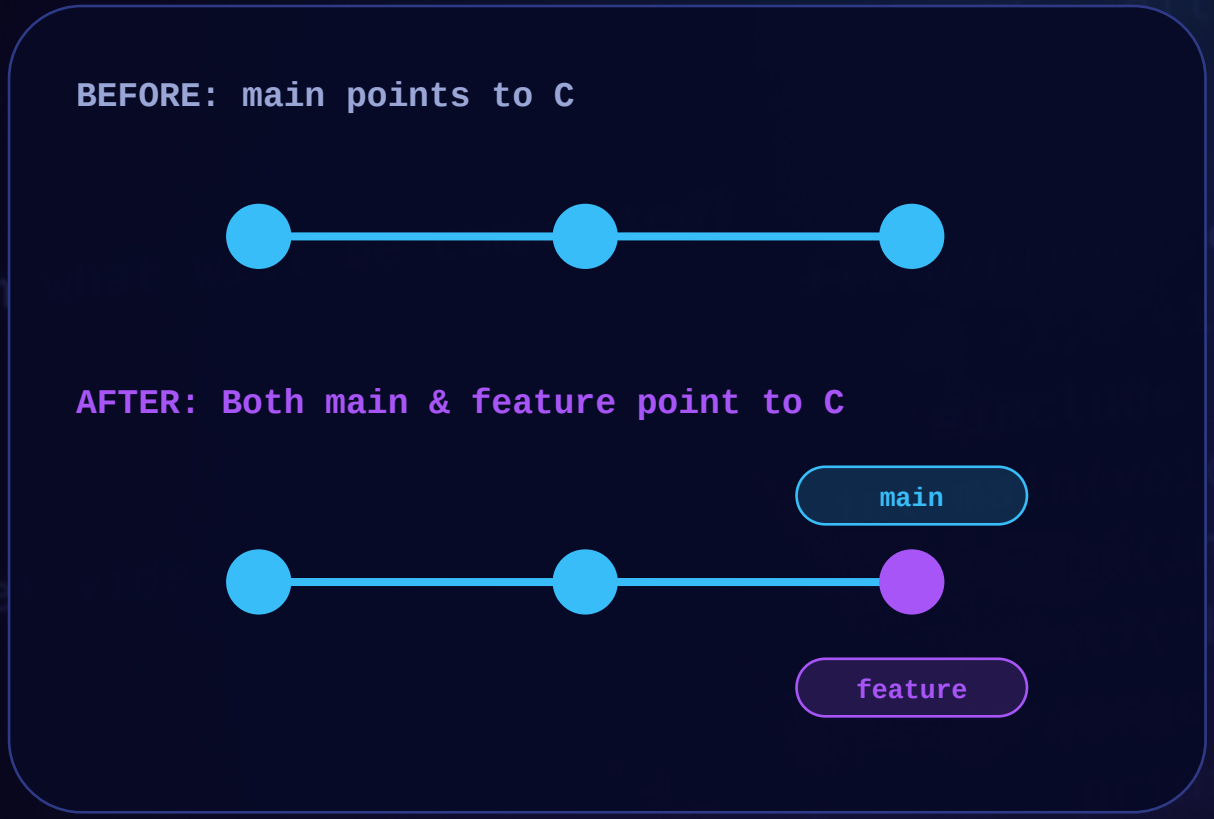
AppsClub

What Is a Branch?

Movable commit reference.

Creating a branch points a new label to the current commit without copying files.

```
> git branch feature
```



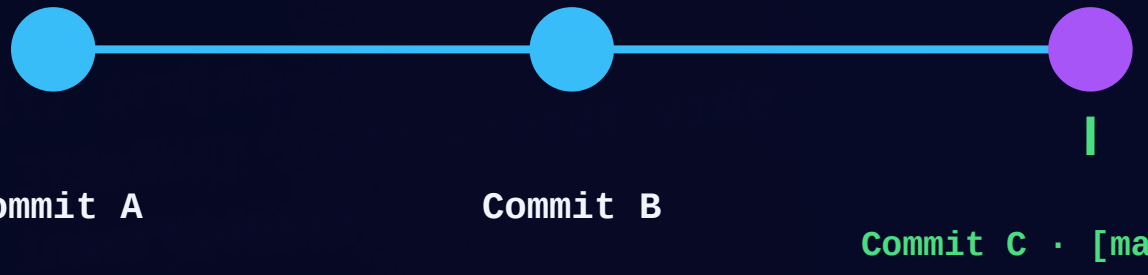
AppsClub

Switching Branches & HEAD

HEAD tracks your active branch. Switching moves HEAD without creating commits.

```
> git switch feature
```

Moves HEAD pointer to feature



AppsClub

Branches Evolve Independently

STEP 1: COMMIT ON FEATURE

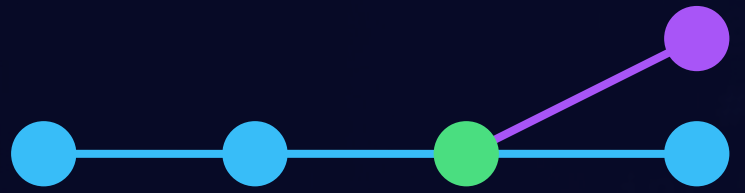
Feature advances to D. Main stays at C.



A — B — C [main] — D [feature]

STEP 2: SWITCH & COMMIT ON MAIN

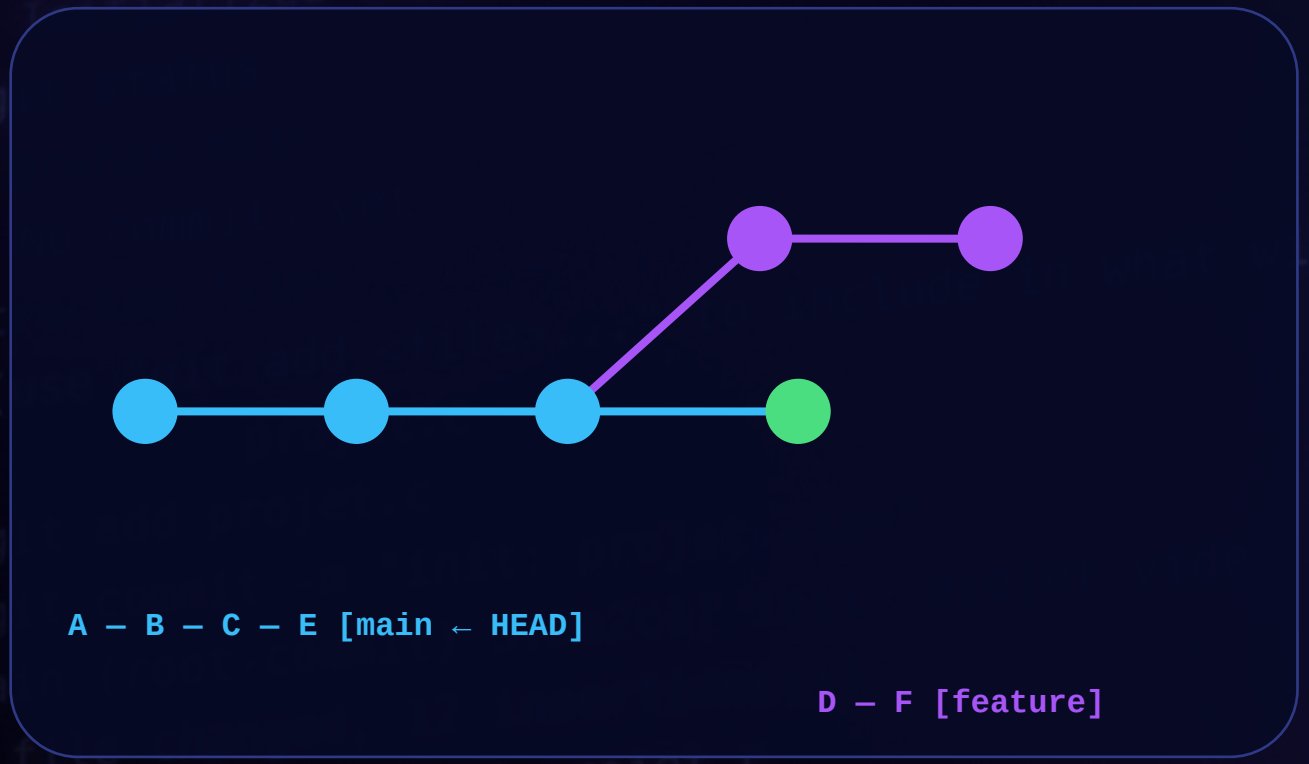
Main advances to E. Histories diverge from C.



Common Ancestor: C · Branches: D & E

AppsClub

Why Do We Need Merging?



Rule: Current branch receives the merge.

Switch to main first before integrating feature work.

```
Destination = Checked-out branch
```


AppsClub

Scenario A: Fast-Forward Merge

```
> git switch main    > git merge feature
```

Before Merge (Linear path)



C [main] -> D -> F [feature]

After Merge (Pointer moves forward)



A — B — C — D — F [main, feature]

AppsClub

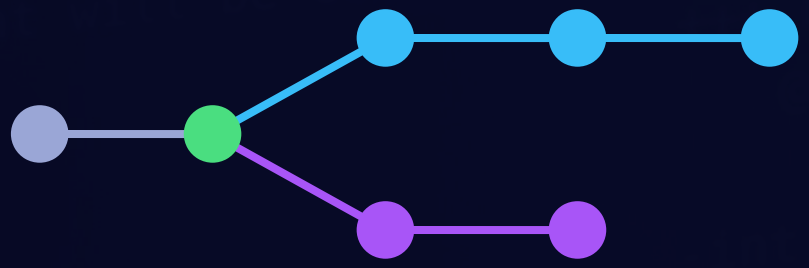
What If Both Branches Changed?

No simple fast-forward possible.

Common Ancestor: Commit B anchors both paths.

Main Branch: Progressed through C, D, E.

Feature Branch: Diverged with commits F, G.



AppsClub

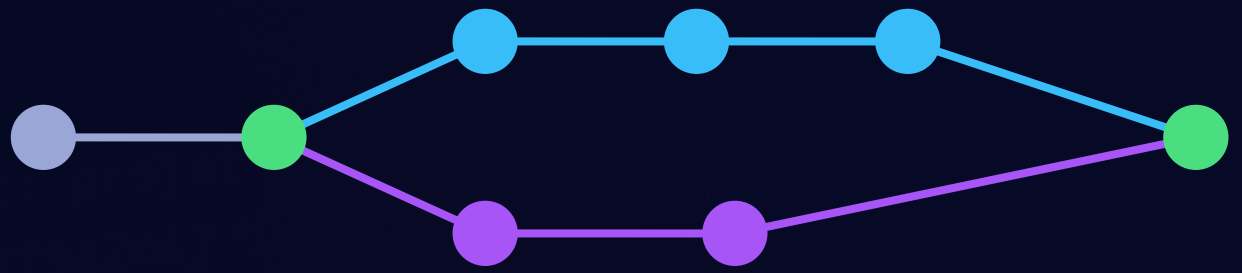
Scenario B: Three-Way Merge

B
1. Common Ancestor

E
2. Main Tip

G
3. Feature Tip

M
New Merge Commit



`Commit M unites parents E & G on [main]`

AppsClub

Clean Merge vs Merge Conflict

Compatible Edits

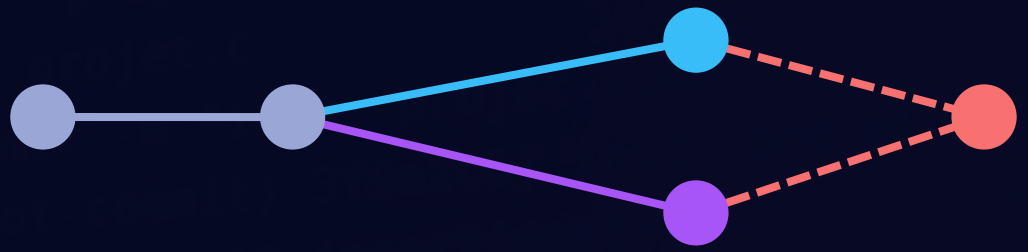
Different lines combine automatically.

Main: "Hello, everyone!"

Modified line 1 on main.

Feature: "Hi, team!"

Competing edit on line 1.



! CONFLICT AT MERGE

Git pauses and asks the developer to choose.

AppsClub

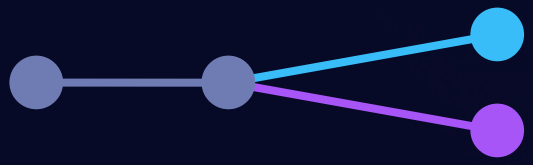
How Does a Conflict Look?

```
<<<<<<< HEAD
Hello, everyone!
=======
Hi, team!
>>>>>>> feature
```

`<<<<<<< HEAD`: Current branch version.

`=======`: Divider between choices.

`>>>>>>> feature`: Incoming branch version.



C [HEAD] vs D [feature] — Paused

AppsClub

How to Resolve a Conflict

STEP 1

Edit File

Keep final text & delete markers.

STEP 2

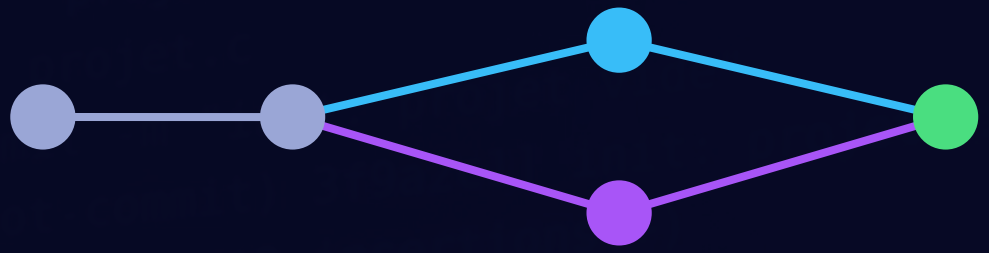
Stage Resolution

```
> git add message.txt
```

STEP 3

Finish Merge

```
> git commit
```



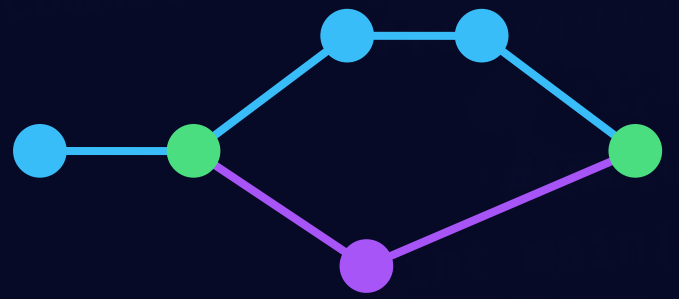
```
✓ MERGE COMPLETE (Commit M)
Resolved: "Hello, everyone!"
```


AppsClub

What Have We Learned?

Concept	Meaning
`Branch`	Movable pointer to a commit
`HEAD`	Current active branch reference
`Fast-forward`	Advance pointer along linear path
`Three-way merge`	Combine diverged histories via M
`Conflict`	Competing changes resolved manually

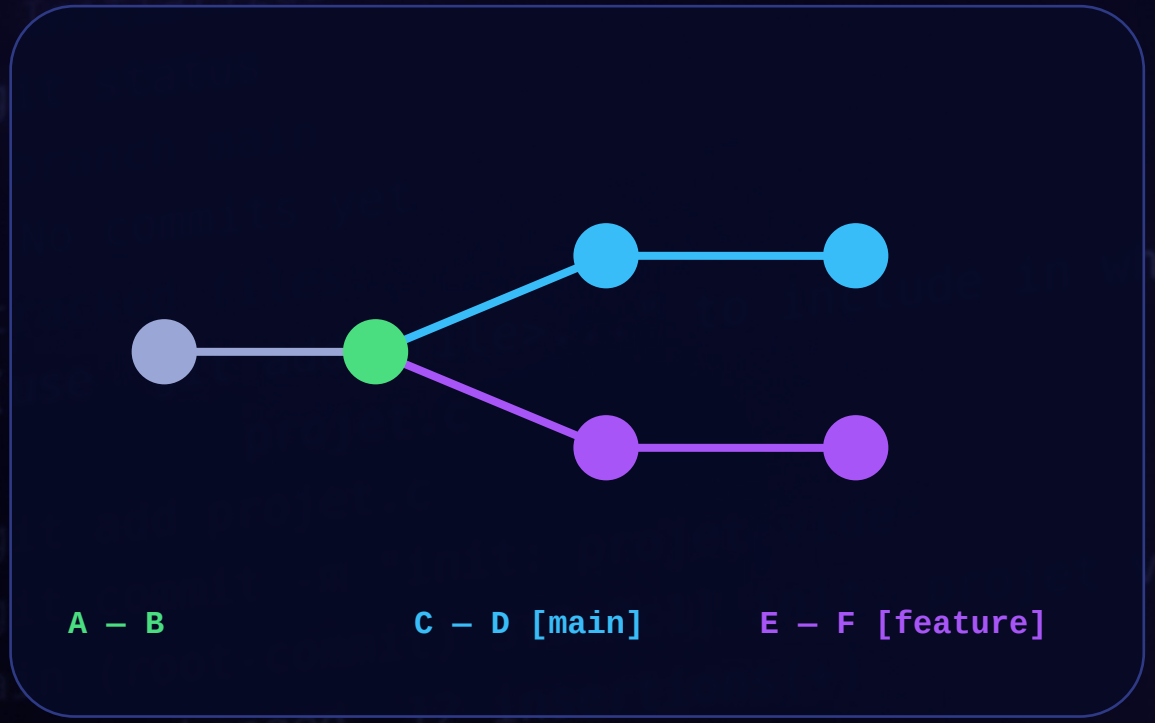
COMPLETE WORKFLOW GRAPH



`A — B — [C—D / E] -> M`

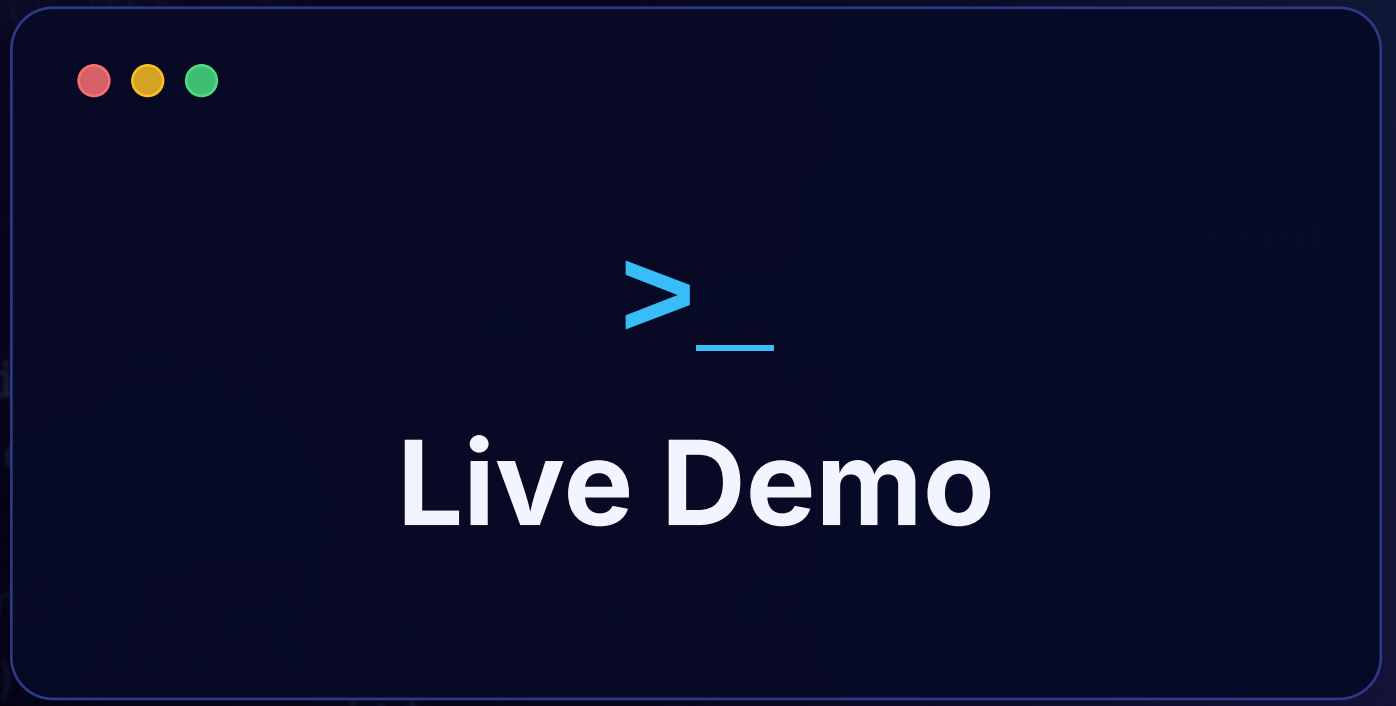
AppsClub

Read the Graph & Predict



1. **Common Ancestor:** Which commit is shared?
2. **Target Branch:** Which branch do we switch to first?
3. **Conflicts:** Is a conflict guaranteed here?
4. **Merge Commit:** Will Git create commit M?
5. **Cleanup:** Does merging delete feature?

AppsClub


>_
Live Demo

>_

Let's practice

AppsClub

AppsClub

Thank you for listening!

Join us again next week!!